Python Coding Task

Section 1: Basic Python

1. FizzBuzz

In [1]:
```
for i in range(1, 51):
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
```

2. Leap Year Logic

In [2]:
```
def is_leap_year(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
```

3. Identity vs Equality

In [3]:
```
L1 = [1, 2, 3]
L2 = [1, 2, 3]

print(L1 == L2)
print(L1 is L2)
```

```
True
False
```

4. Bitwise Swap

In [4]:
```
a = 10
b = 20

a = a ^ b
b = a ^ b
a = a ^ b

print(a, b)
```

```
20 10
```

Section 2: Control Flow & Loops

5. Prime Number Finder

In [5]:
```
n = 2
count = 0

while count < 10:
    prime = True
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            prime = False
            break
    if prime:
        print(n)
        count += 1
    n += 1
```

```
2
3
5
7
11
13
17
19
23
29
```

6. The Loop Breaker

In [6]:
```
for i in range(1, 21):
    if i % 4 == 0:
        pass
    if i == 13:
        continue
    if i == 18:
        break
    print(i)
```

```
1
2
3
4
5
6
7
8
9
10
11
12
14
15
16
17
```

7. Nested Conditionals

In [7]:
```
def grade(marks):
    if marks >= 90:
        return "A"
    elif marks >= 80:
        return "B"
    elif marks >= 70:
        return "C"
    return "Fail"

print(grade(85))
```

```
B
```

8. Reverse an Integer

In [8]:
```
n = 12345
rev = 0

while n > 0:
    rev = rev * 10 + n % 10
    n //= 10

print(rev)
```

```
54321
```

Section 3: Data Structures

9. Runner-Up Score

In [9]:
```
scores = [10, 20, 20, 8, 15]
highest = runner_up = float("-inf")

for score in scores:
    if score > highest:
        runner_up = highest
        highest = score
    elif highest > score > runner_up:
        runner_up = score

print(runner_up)
```

```
15
```

10. Duplicate Removal

In [10]:
```
values = [1, 2, 2, 3, 4, 4]
unique = list(set(values))
unique
```

Out[10]:
```
[1, 2, 3, 4]
```

11. List Intersection

In [11]:
```
L1 = [1, 2, 3, 4]
L2 = [3, 4, 5, 6]

common = [x for x in L1 if x in L2]
common
```

Out[11]:
```
[3, 4]
```

12. Tuple Immutability Test

In [12]:
```
t = (1, 2, 3)

try:
    t[1] = 5
except TypeError as e:
    print(e)
```

```
'tuple' object does not support item assignment
```

Section 4: Dictionaries & Comprehensions

13. Character Frequency Counter

In [13]:
```
text = input("Enter a string: ")
frequency = {}

for char in text:
    frequency[char] = frequency.get(char, 0) + 1

frequency
```

Out[13]:
```
{'H': 1,
 'i': 1,
 ',': 1,
 ' ': 3,
 'I': 1,
 'a': 3,
 'm': 1,
 'T': 1,
 'n': 1,
 's': 1,
 'h': 1,
 'u': 1}
```

14. List Comprehension

In [14]:
```
squares = [x ** 2 for x in range(1, 21) if x % 2 == 0]
squares
```

Out[14]:
```
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400]
```

15. Dictionary Comprehension

In [15]:
```
data = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
result = {k: v for k, v in data.items() if v > 2}
result
```

Out[15]:
```
{'c': 3, 'd': 4}
```